Handling Errors ⚠️

Christopher Martin - *Bowdoin College*
c.martin@bowdoin.edu

Error!!

As you are probably *intimately* familiar, our code can have errors...
Grappling with the machine to understand these errors and diagnosing what causes them is an essential part of programming

Today, we will be discussing different types of errors, how we can interpret them, and how we can recover from them instead of crashing!

```
1 x = 1
2 y = 0
3 z = x / y
```

```
Traceback (most recent call last):
  File "/app/output.s", line 3, in <module>
    z = x / y
        ~~^~~
```

Errors vs Exceptions

Errors

'Fatal' issues that prevent our code from executing at all - mostly typos

These occur when the syntax of your code is not valid Python

```
1 print("Hello world"
```

```
  File "/app/output.s", line 1
    print("Hello world"
         ^
SyntaxError: '(' was never closed
```

Exceptions

Avoidable issues that occur when our code executes - our mistakes

These occur when the syntax of your code is correct but the logic is invalid!

```
1 int("hello")
```

```
Traceback (most recent call last):
  File "/app/output.s", line 1, in <module>
    int("hello")
ValueError: invalid literal for int() with base 10:
'hello'
```

Traceback

Python tries to help us understand the issues caused by our code
These error messages called `tracebacks` tell us what happened and where

Sometimes, these don't tell us exactly what the root issue is and instead just show us a symptom of a bigger issue, that is up to you to decipher!

```
1 grades: dict[str, int] = {"A": 9, "B": 6, "C": 2, "d": 1, "F": 0}
2
3 print(grades["D"])
```

```
Traceback (most recent call last):
  File "/app/output.s", line 3, in <module>
    print(grades["D"])
          ~~~~~~^^^^^
```

There are a lot of different exceptions but most are pretty self-explanatory

We are going to cover the most common ones that I will expect you to be able to recognize

NameError

NameErrors occur when attempting to use something that doesn't exist!

Something here usually refers to variable names and function names

These are usually due to typos!

```
1  my_num = 5
2
3  print(m_num)
```

```
Traceback (most recent call last):
  File "/app/output.s", line 3, in <module>
    print(m_num)
          ^^^^^
NameError: name 'm_num' is not defined. Did you mean: 'my_num'?
```

AttributeError

AttributeErrors occur when attempting to use an invalid variable attribute

Attributes here usually refers to method names

These are usually due to typos or an improper understanding of method names!

```
1 my_list: list[int] = [1, 2, 3]
2
3 my_list.add(4)
```

```
Traceback (most recent call last):
  File "/app/output.s", line 3, in <module>
    my_list.add(4)
    ^^^^^^^^^^^
AttributeError: 'list' object has no attribute 'add'
```

IndexError

IndexErrors occur when attempting to access an invalid numeric position

These are usually due to improper bound checking on lists, tuples, or strings

Does not apply to range-based slicing!

```python
my_list: list[int] = [1, 2, 3]

print(my_list[2:100]) # ✅ Fine!

print(my_list[100])   # ❌ Exception!
```

```
Traceback (most recent call last):
  File "/app/output.s", line 5, in <module>
    print(my_list[100])   # ❌ Exception!
          ~~~~~~~^^^^^
```

KeyError

KeyErrors occur when attempting to access a nonexistent item in an unordered data structure (sets and dictionaries)

These are usually due to improperly assuming an item is in a data structure

Sets and dictionaries both have workarounds!

```
1  commands: set[str] = {"register", "login", "quit"}
2
3  commands.discard("save") # ✅ Fine!
4
5  commands.remove("save")  # ❌ Exception!
```

```
Traceback (most recent call last):
  File "/app/output.s", line 5, in <module>
    commands.remove("save")  # ❌ Exception!
    ^^^^^^^^^^^^^^^^^^^^^^^
```

TypeError

TypeErrors occur when attempting to perform an operation with a type not supported by that operation

These are usually due to an improper understanding of a variables type or the assumptions made by the given operation

Usually fixed by typecasting

```
1 "High" + 5
```

```
Traceback (most recent call last):
  File "/app/output.s", line 1, in <module>
    "High" + 5
    ~~~~~~~^~~
TypeError: can only concatenate str (not "int") to str
```

ValueError

ValueErrors occur when attempting to perform an operation with an invalid value of a proper type not supported by that operation

These are usually due to an improper understanding of the assumptions made by the given operation

Usually requires an intermediate step to ensure a value is valid

```
1 int("hello")
```

```
Traceback (most recent call last):
  File "/app/output.s", line 1, in <module>
    int("hello")
ValueError: invalid literal for int() with base 10: 'hello'
```

ZeroDivisionError

ZeroDivisionErrors occur when attempting divide by zero

These are usually due to an improper checking to ensure a value is not zero

Usually requires an intermediate step to ensure a value is valid

```
1 zero = 0
2
3 5 / zero  # ❌ Exception!
4 4 % zero  # ❌ Exception!
5 3 // zero # ❌ Exception!
```

```
Traceback (most recent call last):
  File "/app/output.s", line 3, in <module>
    5 / zero  # ❌ Exception!
    ~~^~~~~
```

Phew, that was a lot!

You're telling me there are even more?!

Unfortunately, yes...

Luckily, those are the only ones I expect you to know

Now, let’s talk about how we can prevent these exceptions from crashing our programs even if they do occur!

Try Except

A `try except` block allows us to safely try to execute some code if we know an exception may happen in advance
If the exception does happen, we start executing the except block instead!

Their syntax is as follows:

```
try:
  Code we want to execute that may cause an exception
  ...
except EXCEPTION_TYPE:
  Code we want to execute instead of crashing
  ...
```

- **`EXCEPTION_TYPE` can be any type of exception!**
  The code inside the except will run instead of crashing your program, once it is complete your program will continue executing like normal!

Here is an example of how we can use them!

```python
# Try changing the numbers!
x = 5
y = 0

try:
  print(f"{x} / {y} = {x / y}")
except ZeroDivisionError:
  print(f"{x} / {y} is undefined!")

print("\nThanks for using the program!")
```

```
5 / 0 is undefined!
Thanks for using the program!
```

Try Except

You can add multiple excepts if multiple exceptions can occur!

```
1  try:
2    x = float(input("Enter a value for x: "))
3    y = float(input("Enter a value for y: "))
4    print(f"{x} / {y} = {x / y}")
5  except ValueError:
6    print("That isn't a valid number!")
7  except ZeroDivisionError:
8    print(f"{x} / {y} is undefined!")
9
10 print("\nThanks for using the program!")
```

Raising Exceptions

You can choose to manually 'raise' any type of exception!

This is done to inform other developers that they are using your code incorrectly

```
1  raise KeyError("This is the message that will show up in the traceback!")
```

```
Traceback (most recent call last):
  File "/app/output.s", line 1, in <module>
    raise KeyError("This is the message that will show up in the traceback!")
KeyError: 'This is the message that will show up in the traceback!'
```

Typically, we use this when our functions don’t have something valid to return for given arguments

Raising Exceptions

Here is an example:

```python
def sum(n: int) -> int:
  if n < 0:
    raise ValueError("n cannot be negative!")

  return (n * (n + 1)) / 2

print(sum(5))
```

```
15.0
```

Try Except Finally

We can add a `finally` clause to a `try except` block to ensure code runs even if none of the excepts match

The finally code will run if an anticipated exception, unanticipated exception, or no exception occurs!

The syntax is as follows:

```
try:
  Code we want to execute that may cause an exception
  ...
except EXCEPTION_TYPE:
  Code we want to execute instead of crashing
  ...
finally:
  Code we want to execute if an exception happens or not
  ...
```

Try Except Finally

Here is an example of using a finally clause!

```python
x = 5
y = 2

try:
  print(f"{x} / {y} = {x / y}")
except ZeroDivisionError:
  print(f"{x} / {y} is undefined!")
finally:
  print("Exitting the try-except!")

print("\nThanks for using the program!")
```

```
5 / 2 = 2.5
Exitting the try-except!

Thanks for using the program!
```